

Project #8 — Dominator Problem

Divide & Conquer Vs Sorting-based dominator detection

Algorithm 1 — Divide & Conquer (Recursive)

Idea: Split the array in half recursively until each part has one element. Merge back up by checking which candidate is dominant in the combined range.

Pseudo-Code

Divide & Conquer Dominator — Pseudo-Code

```
FUNCTION countInRange(A[], lowestIndex, highestIndex, val):
    freq <- 0
    FOR i <- lowestIndex TO highestIndex DO
        IF A[i] = val THEN freq <- freq + 1
    RETURN freq
END FUNCTION

FUNCTION dominatorDC(A[], lowestIndex, highestIndex):
    IF lowestIndex = highestIndex THEN RETURN A[lowestIndex]

    mid <- (lowestIndex + highestIndex) / 2
    leftCand <- dominatorDC(A, lowestIndex, mid)
    rightCand <- dominatorDC(A, mid+1, highestIndex)

    IF leftCand = rightCand THEN RETURN leftCand

    size <- highestIndex - lowestIndex + 1

    IF leftCand != NO_CANDIDATE THEN
        lCount <- countInRange(A, lowestIndex, highestIndex, leftCand)
        IF lCount > size/2 THEN RETURN leftCand
    END IF

    IF rightCand != NO_CANDIDATE THEN
        rCount <- countInRange(A, lowestIndex, highestIndex, rightCand)
        IF rCount > size/2 THEN RETURN rightCand
    END IF

    RETURN NO_CANDIDATE
END FUNCTION

FUNCTION findDominator_DC(A[], n):
    IF n = 0 THEN RETURN -1
    candidate <- dominatorDC(A, 0, n-1)
    IF candidate = NO_CANDIDATE THEN RETURN -1

    firstIndex <- -1
    PRINT 'Dominator value: ' candidate
    PRINT 'Dominator indices: '
    FOR i <- 0 TO n-1 DO
        IF A[i] = candidate THEN
            IF firstIndex = -1 THEN firstIndex <- i
            PRINT i
        END IF
    END FOR
    RETURN firstIndex
```

```
END FUNCTION
```

```
MAIN:
```

```
  READ N
```

```
  READ A[0..N-1]
```

```
  result <- findDominator_DC(A, N)
```

```
  IF result = -1 THEN PRINT 'No Dominator found'
```

```
  ELSE PRINT 'First index: ' result
```

Complexity Analysis

Recurrence Relation:

Recurrence Relation — Algorithm 1

$$T(n) = 2 * T(n/2) + n$$

Because:

- We split into 2 subproblems of size $n/2$
- `countInRange()` does n work at each merge level

Solving with Master Theorem:

Master Theorem Solution

$$T(n) = a * T(n/b) + f(n)$$

$a = 2$ (two recursive calls)

$b = 2$ (array split in half)

$f(n) = n$

$$\log_b(a) = \log_2(2) = 1$$

$f(n) = n = O(n^1) \rightarrow$ Case 2 of Master Theorem

Result: $T(n) = O(n \log n)$

Space: recursion depth = $\log_2(n) \rightarrow O(\log n)$

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$

Best Case: $O(n \log n)$ | **Worst Case:** $O(n \log n)$ | **Average:** $O(n \log n)$

Algorithm 2 — Insertion Sort (Non-Recursive)

Idea: Sort a copy of the array using Insertion Sort, then check if $\text{sorted}[i]$ equals $\text{sorted}[i + n/2]$. If equal, that value appears more than half the time and is the Dominator.

Pseudo-Code

Insertion Sort Dominator — Pseudo-Code

```
FUNCTION insertionSort(arr[], N):
  FOR i <- 1 TO N-1 DO
    key <- arr[i]
    j <- i - 1
    WHILE j >= 0 AND arr[j] > key DO
      arr[j+1] <- arr[j]
      j <- j - 1
    END WHILE
    arr[j+1] <- key
  END FOR
END FUNCTION

FUNCTION printDominatorIndices(original[], length):
  IF length = 0 THEN RETURN -1
  IF length = 1 THEN
    PRINT 'Dominator found at index: 0'
    RETURN 0
  END IF

  sorted <- COPY of original
  CALL insertionSort(sorted, length)

  half <- length / 2
  candidate <- NOT_FOUND
  found <- false

  FOR i <- 0 TO length - half - 1 DO
    IF sorted[i] = sorted[i + half] THEN
      candidate <- sorted[i]
      found <- true
      BREAK
    END IF
  END FOR

  IF NOT found THEN
    PRINT 'No dominator found'
    RETURN -1
  END IF

  firstIndex <- -1
  PRINT 'Dominator found at indices: '
  FOR i <- 0 TO length-1 DO
    IF original[i] = candidate THEN
      IF firstIndex = -1 THEN firstIndex <- i
      PRINT i
    END IF
```

```
    END FOR
    PRINT 'Dominator value: ' candidate
    RETURN firstIndex
END FUNCTION

MAIN:
    READ N
    READ A[0..N-1]
    result <- printDominatorIndices(A, N)
    IF result = -1 THEN PRINT 'No dominator. Returning -1'
    ELSE PRINT 'Returned index: ' result
```

Complexity Analysis

Step-by-step count:

Phase	Operations	Explanation
Copy array	n	One pass to copy original into sorted vector
Insertion Sort	$O(n^2)$	Each element may shift through all sorted elements
Find candidate	$\leq n$	One pass: compare <code>sorted[i]</code> with <code>sorted[i + half]</code>
Print indices	n	One pass over original array to print all indices
Total	$O(n^2)$	Dominated by the Insertion Sort step

Recurrence Relation:

Recurrence Relation — Algorithm 2

$T(n) = T(n-1) + n$ (Insertion Sort)
 $T(1) = 1$

Solving:

$$\begin{aligned}T(n) &= T(n-1) + n \\&= T(n-2) + (n-1) + n \\&= T(n-3) + (n-2) + (n-1) + n \\&= T(1) + 2 + 3 + \dots + n \\&= 1 + n(n+1)/2 - 1 \\&= n(n+1)/2 \\&= O(n^2)\end{aligned}$$

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Best Case: $O(n)$ | **Worst Case:** $O(n^2)$ | **Average:** $O(n^2)$

Comparison

Criterion	Algorithm 1 — Divide & Conquer	Algorithm 2 — Insertion Sort
Paradigm	Recursive Splitting	Sort-based Linear Scan
Time Complexity	$O(n \log n)$	$O(n^2)$
Space Complexity	$O(\log n)$ stack	$O(n)$
Approach	Divide & Conquer	Insertion Sort + Search
Input	User entered array	User entered array
Returns	First index + all idx	First index + all idx
Preferred	Yes (faster)	No (slower)

Conclusion:

Divide & Conquer is the faster choice at $O(n \log n)$ time and $O(\log n)$ space.

Insertion Sort is simpler to understand but slower at $O(n^2)$ due to the sorting step.